How to Write scenarioWorldManager.ts Logic

Step-by-Step Logic Breakdown

---

1. Setup - Initialize ECS and Storage

typescript

```typescript
import { EntityManager } from './ecsManager';
import { Position, Rotation, Color } from './components';
import { Entity } from './types';

// Create ONE EntityManager for the scenario world
const entityManager = new EntityManager();

// Map to track: object-id (string) -> entity (number)
// Example: "obj-123" -> 5
const objectMap = new Map<string, Entity>();
```

Why?

- `entityManager` = Your ECS engine that manages all entities
- `objectMap` = We need to remember which object-id maps to which entity
- This lets us find entities by their friendly ID (like "obj-123")

---

2. Create Scenario World - Reset Everything

typescript

```
export function createScenarioWorld(): ScenarioWorld {
  // Clear the map (removes all ID -> entity mappings)
  objectMap.clear();

  // Note: We don't need to clear entityManager
  // because we'll just stop tracking old entities
  // and they become unreachable

  return {
    objects: []  // Start with empty world
  };
}
```

Logic Flow:

1. User calls createScenarioWorld()
2. Clear objectMap (forget all object IDs)
3. Old entities still exist in EntityManager but we lost their IDs
4. Return empty world
5. Next addObject() will create fresh entities

Why clear objectMap?

- Makes old entities unreachable
- Fresh start for new scenario
- Simple and effective

---

3. Add Object - The Core Logic

typescript

```
export function addObject(
  position: { x: number; y: number; z: number },
  rotation: { x: number; y: number; z: number },
  color: string
): ScenarioObject {

  // STEP 1: Generate unique ID
  const objectId = `obj-${Date.now()}-${Math.random().toString(36).substr(2, 9)}`;
  // Example: "obj-1729932450-k2j3h4l5"

  // STEP 2: Create entity (gets a numeric ID)
  const entity = entityManager.createEntity();
  // entity = 0, 1, 2, 3... (incrementing number)

  // STEP 3: Create and attach Position component
  entityManager.addComponent(
    entity,
    new Position(position.x, position.y, position.z)
  );

  // STEP 4: Create and attach Rotation component
  entityManager.addComponent(
    entity,
    new Rotation(rotation.x, rotation.y, rotation.z)
  );

  // STEP 5: Create and attach Color component
  entityManager.addComponent(
    entity,
    new Color(color)
  );

  // STEP 6: Remember the mapping
  objectMap.set(objectId, entity);
  // Now we can find this entity later using objectId

  // STEP 7: Convert to object format and return
  const obj = entityToObject(objectId, entity);
  if (!obj) throw new Error('Failed to create object');

  return obj;
}
```

Visual Flow:

```
Input:
  position: {x: 5, y: 0, z: 10}
  rotation: {x: 0, y: 45, z: 0}
  color: "#FF0000"

↓

Generate ID: "obj-1729932450-abc123"

↓

EntityManager.createEntity() → returns entity: 5

↓

Entity 5:
  ├─ Position{x: 5, y: 0, z: 10}
  ├─ Rotation{x: 0, y: 45, z: 0}
  └─ Color{value: "#FF0000"}

↓

objectMap.set("obj-1729932450-abc123", 5)

↓

Return: {
  id: "obj-1729932450-abc123",
  position: {x: 5, y: 0, z: 10},
  rotation: {x: 0, y: 45, z: 0},
  color: "#FF0000"
}
```

---

4. Entity to Object Conversion - Helper Function

typescript

```
function entityToObject(objectId: string, entity: Entity): ScenarioObject | null {

  // STEP 1: Get Position component from entity
  const pos = entityManager.getComponent(entity, Position);

  // STEP 2: Get Rotation component from entity
  const rot = entityManager.getComponent(entity, Rotation);

  // STEP 3: Get Color component from entity
  const col = entityManager.getComponent(entity, Color);

  // STEP 4: Check if all components exist
  if (!pos || !rot || !col) return null;

  // STEP 5: Build object in API format
  return {
    id: objectId,                          // String ID
    position: { x: pos.x, y: pos.y, z: pos.z }, // Extract from component
    rotation: { x: rot.x, y: rot.y, z: rot.z }, // Extract from component
    color: col.value                       // Extract from component
  };
}
```

Why this function?

- ECS stores data as components
- API needs data as plain objects
- This function converts: Entity + Components → Object

Visual:

```
Entity 5 has:
  Position{x: 5, y: 0, z: 10}
  Rotation{x: 0, y: 45, z: 0}
  Color{value: "#FF0000"}

↓ entityToObject("obj-123", 5)

{
  id: "obj-123",
  position: {x: 5, y: 0, z: 10},
  rotation: {x: 0, y: 45, z: 0},
  color: "#FF0000"
}
```

---

5. Remove Object - Delete by ID

typescript

```typescript
export function removeObject(objectId: string): boolean {

  // STEP 1: Find entity using the ID
  const entity = objectMap.get(objectId);

  // STEP 2: Check if entity exists
  if (entity === undefined) return false;

  // STEP 3: Remove entity from ECS
  // This removes the entity AND all its components
  entityManager.removeEntity(entity);

  // STEP 4: Remove from our map
  objectMap.delete(objectId);

  // STEP 5: Success
  return true;
}
```

Logic Flow:

Input: "obj-1729932450-abc123"

↓

objectMap.get("obj-1729932450-abc123") → returns entity: 5

↓

entityManager.removeEntity(5)
- Removes entity 5
- Removes all its components (Position, Rotation, Color)

↓

objectMap.delete("obj-1729932450-abc123")
- Removes mapping

↓

Return: true (success)

If object not found:

Input: "obj-999-notfound"

↓

objectMap.get("obj-999-notfound") → returns undefined

↓

Return: false (not found)

---

6. Update Object - Modify Components

typescript

```typescript
export function updateObject(
  objectId: string,
  position: { x: number; y: number; z: number },
  rotation: { x: number; y: number; z: number },
  color: string
): ScenarioObject | null {

  // STEP 1: Find entity
  const entity = objectMap.get(objectId);
  if (entity === undefined) return null;

  // STEP 2: Get components
  const pos = entityManager.getComponent(entity, Position);
  const rot = entityManager.getComponent(entity, Rotation);
  const col = entityManager.getComponent(entity, Color);

  // STEP 3: Check components exist
  if (!pos || !rot || !col) return null;

  // STEP 4: Modify Position component IN-PLACE
  pos.x = position.x;
  pos.y = position.y;
  pos.z = position.z;

  // STEP 5: Modify Rotation component IN-PLACE
  rot.x = rotation.x;
  rot.y = rotation.y;
  rot.z = rotation.z;

  // STEP 6: Modify Color component IN-PLACE
  col.value = color;

  // STEP 7: Convert and return updated object
  return entityToObject(objectId, entity);
}
```

Key Insight: IN-PLACE MODIFICATION

typescript

```
// We DON'T do this:
entityManager.removeComponent(entity, Position);
entityManager.addComponent(entity, new Position(x, y, z));

// We DO this (direct modification):
const pos = entityManager.getComponent(entity, Position);
pos.x = newX;
pos.y = newY;
pos.z = newZ;
```

Why?

- Components are objects (reference types)
- We can modify them directly
- Faster and simpler
- ECS automatically sees the changes

Visual Flow:

Input:
  objectId: "obj-123"
  position: {x: 10, y: 2, z: 15}
  rotation: {x: 0, y: 90, z: 0}
  color: "#00FF00"

↓

Find entity: objectMap.get("obj-123") → 5

↓

Get components from entity 5:
  pos = Position{x: 5, y: 0, z: 10}
  rot = Rotation{x: 0, y: 45, z: 0}
  col = Color{value: "#FF0000"}

↓

Modify in-place:
  pos.x = 10, pos.y = 2, pos.z = 15
  rot.x = 0, rot.y = 90, rot.z = 0
  col.value = "#00FF00"

↓

Now entity 5 has:
  Position{x: 10, y: 2, z: 15} ← changed!
  Rotation{x: 0, y: 90, z: 0}   ← changed!
  Color{value: "#00FF00"}   ← changed!

↓

Return updated object

---

7. Get Scenario World - Return All Objects

typescript

```
export function getScenarioWorld(): ScenarioWorld {
  const objects: ScenarioObject[] = [];

  // STEP 1: Loop through all tracked objects
  for (const [objectId, entity] of objectMap.entries()) {

    // STEP 2: Convert each entity to object format
    const obj = entityToObject(objectId, entity);

    // STEP 3: Add to array (if valid)
    if (obj) objects.push(obj);
  }

  // STEP 4: Return world with all objects
  return { objects };
}
```

Visual:

objectMap:
  "obj-123" -> 5
  "obj-456" -> 7
  "obj-789" -> 9

↓

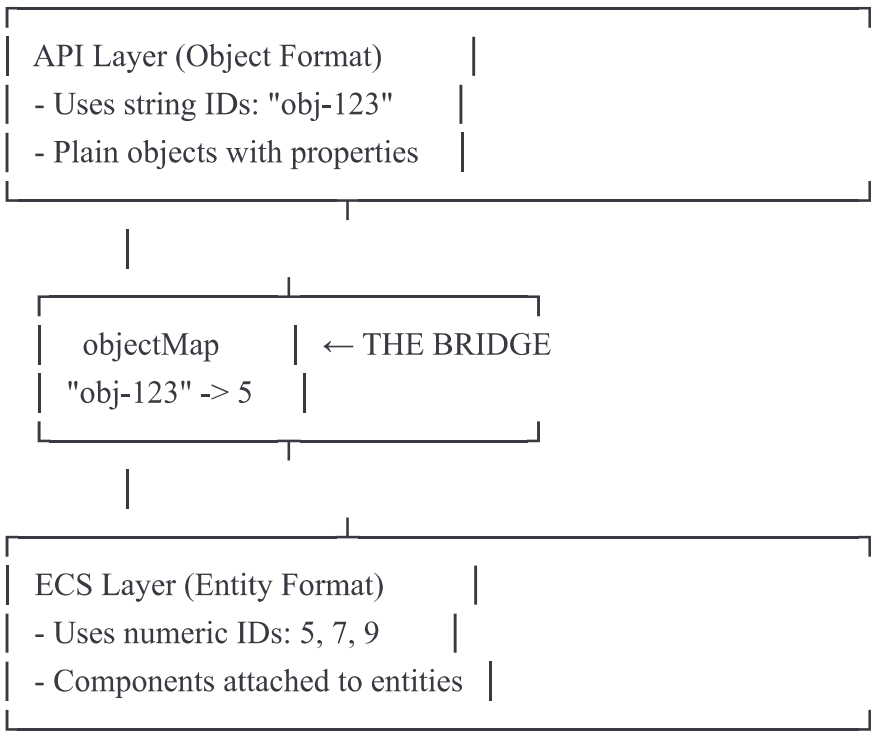
Loop:
  1. entityToObject("obj-123", 5) → {id: "obj-123", ...}
  2. entityToObject("obj-456", 7) → {id: "obj-456", ...}
  3. entityToObject("obj-789", 9) → {id: "obj-789", ...}

↓

Return:
{
  objects: [
    {id: "obj-123", position: {...}, rotation: {...}, color: "..."},
    {id: "obj-456", position: {...}, rotation: {...}, color: "..."},
    {id: "obj-789", position: {...}, rotation: {...}, color: "..."}
  ]
}

Complete Mental Model

The Two Layers:



The Flow:

Adding Object:

API → Generate ID → Create Entity → Add Components → Map ID to Entity → Return Object

Updating Object:

API → Find Entity by ID → Get Components → Modify Components → Return Object

Deleting Object:

API → Find Entity by ID → Remove Entity → Remove from Map → Return Success

Getting World:

API → Loop Map → Convert Each Entity → Return Array of Objects

---

Key Concepts

1. The Map is Critical

typescript

```
objectMap.set("obj-123", 5);
// Without this, we can't find entity 5 later!
```

2. Components are References

typescript

```
const pos = entityManager.getComponent(entity, Position);
pos.x = 10;  // This modifies the actual component!
```

3. ID Generation

typescript

```
`obj-${Date.now()}-${Math.random().toString(36).substr(2, 9)}`
// Ensures unique IDs even with concurrent requests
```

4. Conversion Function

typescript

```
entityToObject()  // Always needed to return data to API
// Converts: ECS format → API format
```

5. Error Handling

typescript

```
if (entity === undefined) return null;
// Always check if entity exists before using it
```

Common Patterns

Pattern 1: Find then Modify

typescript

```
const entity = objectMap.get(id);      // Find
if (!entity) return null;              // Check
const component = getComponent(...);   // Get
component.x = newValue;                // Modify
```

Pattern 2: Create then Track

typescript

```
const entity = createEntity();         // Create
addComponent(entity, new Position()); // Setup
objectMap.set(id, entity);             // Track
```

Pattern 3: Convert for Return

typescript

```
const entity = /* ... do work ... */
return entityToObject(id, entity);    // Always convert before returning
```

Summary

The logic is simple:

1. **Store**: Map string IDs to numeric entities
2. **Create**: Make entity, add components, track ID
3. **Read**: Find entity, get components, convert format
4. **Update**: Find entity, modify components in-place

5. **Delete**: Find entity, remove from ECS and map

The hard part is understanding the **two-layer system** (API layer vs ECS layer) and how the **objectMap bridges them**!